IoT Home Air Quality Monitor — Final Project Plan

Software Engineering Project 777 — Complete Plan (Academic + 4-Stage Build)

> **How this document works:** Sections 1–7 are the **academic specification** (use cases, ERD, requirements, API, validation — what you'd show your teacher).
>
> Sections 8–12 are the **4-stage build plan** that progresses through Lab 9 → Lab 10 → Activity 5 → Lab 19 (Docker), with testing checklists for each stage.

---

0. Project Summary

A desktop application that monitors air quality (PM2.5 and CO2 levels) in different rooms of a home using IoT sensors. The system continuously logs sensor data, displays real-time information on a dashboard, and automatically generates alerts when pollutant levels exceed safe thresholds. The system supports role-based access (Admin / Operator), CSV export, and a background simulator that mimics real IoT sensor traffic.

Tech Stack

Layer	Technology	Purpose
Database	SQLite	Local data storage
Backend	FastAPI (Python)	REST API server
Frontend	Flet (Python)	Desktop UI
Auth	bcrypt + tokens	Password hashing and session tokens
Simulator	Python script	Fake IoT sensor data generator

---

1. Use Case Diagram

1.1 Actors

Actor	Type	Description
User — Admin	Primary Human	Full access: monitors air quality, manages devices, configures thresholds, views all reports
User — Operator	Primary Human	Read-only access: views dashboard, reads alerts, exports reports (cannot edit devices/settings)
IoT Sensor	External System	Sends PM2.5 / CO2 readings via `POST /readings` (real sensor or simulator)

1.2 Use Cases

ID	Use Case	Actor(s)	Description
UC-01	Sign In / Authenticate	Admin, Operator	User enters credentials; system validates and grants role-based session
UC-02	Sign Out	Admin, Operator	User ends session; system clears token
UC-03	View Dashboard	Admin, Operator	View summary cards (avg PM2.5/CO2, device count, alert count) + live line chart
UC-04	Manage Devices (CRUD)	Admin only	Add, edit, delete IoT devices; view in paginated/searchable DataTable

ID	Use Case	Actor(s)	Description
UC-05	View Readings Table	Admin, Operator	Browse all pollutant readings with search, sort, and pagination
UC-06	Configure Thresholds	Admin only	Set PM2.5 and CO2 alert limits per device
UC-07	Manage Alerts	Admin, Operator	View alert history, filter by type/room, acknowledge alerts
UC-08	Export Reports	Admin, Operator	Download readings or alert history as CSV
UC-09	Send Reading Data	IoT Sensor	POST sensor values; system auto-checks thresholds and creates alerts

---

2. Entity Relationship Diagram (ERD)

2.1 Entities

Entity	Key Attributes	Notes
`rooms`	room_id (PK), name, floor, created_at	Physical locations of devices
`devices`	device_id (PK), model, status, room_id (FK), installed_at, pm25_threshold, co2_threshold	IoT sensors
`readings`	reading_id (PK AUTOINCREMENT), device_id (FK), pm25, co2, temperature, humidity, timestamp	Pollutant measurements

Entity	Key Attributes	Notes
`alerts`	alert_id (PK AUTOINCREMENT), device_id (FK), reading_id (FK), alert_type, value, threshold, timestamp, acknowledged	Auto-generated warnings
`users`	user_id (PK AUTOINCREMENT), username (UNIQUE), password_hash, role (admin/operator), created_at	Login accounts

2.2 Relationships

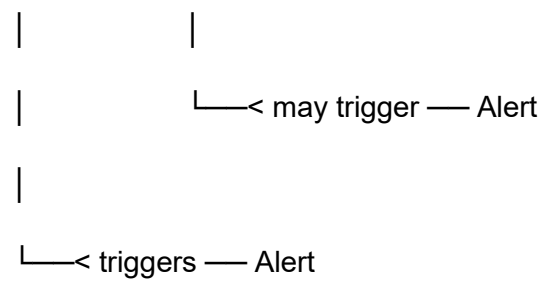
From	To	Cardinality	Description
Room	Device	1 : N	One room can contain many devices
Device	Reading	1 : N	One device generates many readings
Reading	Alert	1 : 0..N (max 2)	A single reading may trigger up to 2 alerts (one for PM2.5 AND one for CO2)
Device	Alert	1 : N	One device can have many alerts over time
User	—	—	Standalone auth table; no FK into other tables

2.3 Visual Relationship Map

Room (1) ——— has many ———< Device (many)

|

├——< has many —— Reading (many)



2.4 SQLite Schema

```
CREATE TABLE rooms (
    room_id    TEXT PRIMARY KEY,
    name       TEXT NOT NULL,
    floor      TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now'))
);

CREATE TABLE devices (
    device_id      TEXT PRIMARY KEY,
    model          TEXT NOT NULL,
    status         TEXT NOT NULL DEFAULT 'active',
    room_id        TEXT NOT NULL,
    installed_at   TEXT DEFAULT (datetime('now')),
    pm25_threshold REAL DEFAULT 35.0,
    co2_threshold  REAL DEFAULT 1000.0,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id) ON DELETE RESTRICT
);

CREATE TABLE readings (
```

```
    reading_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id   TEXT NOT NULL,
    pm25        REAL NOT NULL,
    co2         REAL NOT NULL,
    temperature REAL,
    humidity    REAL,
    timestamp   TEXT DEFAULT (datetime('now')),
    FOREIGN KEY (device_id) REFERENCES devices(device_id) ON DELETE CASCADE
);
CREATE TABLE alerts (
    alert_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id    TEXT NOT NULL,
    reading_id   INTEGER NOT NULL,
    alert_type   TEXT NOT NULL CHECK(alert_type IN ('pm25','co2')),
    value        REAL NOT NULL,
    threshold    REAL NOT NULL,
    timestamp    TEXT DEFAULT (datetime('now')),
    acknowledged INTEGER DEFAULT 0,
    FOREIGN KEY (device_id)  REFERENCES devices(device_id) ON DELETE CASCADE,
    FOREIGN KEY (reading_id) REFERENCES readings(reading_id) ON DELETE CASCADE
);
CREATE TABLE users (
```

```
    user_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role          TEXT NOT NULL CHECK(role IN ('admin','operator')),
    created_at    TEXT DEFAULT (datetime('now'))
);
```

---

3. Requirements Checklist

3.1 Functional Requirements

ID	Requirement	Lab Source	Flet Component	Stage
FR-01	User can sign in with validation — check both fields non-empty; on success, read role from API response and redirect (Admin=4 tabs, Operator=Settings hidden)	General	TextField, Snackbar, NavigationBar	3
FR-02	Dashboard summary cards (active devices, avg PM2.5/CO2, alert count)	General	Container, Row, Text	2
FR-03	Dashboard line chart of readings over time	General	LineChart	3

ID	Requirement	Lab Source	Flet Component	Stage
FR-04	View all devices/readings in DataTable	Lab #9	DataTable, DataColumn, DataRow	1
FR-05	Paginate through records (10 per page)	Activity #5	Row of page buttons, IconButton	1
FR-06	Search records by text (live filter)	Lab #10 + Activity #5	TextField with on_change	1
FR-07	Sort records by any column	Activity #5	Dropdown	1
FR-08	Add new record via form (POST)	Lab #9	TextField, ElevatedButton, Snackbar	2
FR-09	Edit record via modal dialog (PUT) — pre-filled, validated	Lab #10	AlertDialog, TextField	3
FR-10	Delete record with confirmation dialog (DELETE)	Lab #10	IconButton, AlertDialog	3
FR-11	NavigationBar switches between pages	Lab #9	NavigationBar	2
FR-12	AppBar on every view	General	AppBar	2
FR-13	BottomAppBar with quick actions	General	BottomAppBar	2
FR-14	MenuBar with File/Edit/View options	General	MenuBar	3

ID	Requirement	Lab Source	Flet Component	Stage
FR-15	BottomSheet for quick-add reading	General	BottomSheet	2
FR-16	Snackbar for all success/error notifications	Lab #9 + #10	SnackBar	2
FR-17	Alert threshold configuration per device	General	TextField, Dropdown	3
FR-18	Alert log with acknowledge action	General	DataTable with IconButton	3
FR-19	Auto-refresh dashboard every 5 seconds	General	asyncio.sleep + load_data()	3
FR-20	Data persists across restarts (SQLite)	Lab #9	sqlite3	1
FR-21	Export readings/alerts as CSV download	General	ElevatedButton + Python csv module	3
FR-22	Background simulator generates sensor readings automatically	General	Python script (`requests` + `time.sleep`)	3

3.2 Non-Functional Requirements

ID	Requirement
NFR-01	Input validation on both client (Flet) and server (Pydantic)

ID	Requirement
NFR-02	SQL injection prevention via parameterized queries and column whitelist for sort
NFR-03	API responses follow consistent JSON format `{data, error, total}`
NFR-04	Pagination capped at max 100 rows per request
NFR-05	Frontend gracefully handles API connection errors (try/except + Snackbar)
NFR-06	Passwords stored as bcrypt hashes only — never plain text in DB or logs
NFR-07	CORS configured in FastAPI to only allow the Flet client origin

3.3 Security & Authentication

Role-Based Access Control (RBAC): The system implements two roles: `admin` and `operator`.

- **Admin** has full access to all features, including device CRUD, threshold configuration, and full data access.
- **Operator** has read-only access to dashboards, readings, alerts, and CSV export. Cannot edit devices or settings.

Authorization is enforced on the backend for every protected API endpoint. The system verifies the user's role (extracted from the auth token) before executing any action. Unauthorized requests return **HTTP 403 Forbidden**.

Token-Based Authentication & Session Handling:

- On successful `POST /auth/login`, the backend returns a session token + the user's role.
- The frontend stores the token in application state (Python variable) and sends it on every request via `Authorization: Bearer <token>`.
- The backend validates the token on each request. Missing / invalid / expired tokens return **HTTP 401 Unauthorized**.
- On `POST /auth/logout`, the token is invalidated from the frontend state, ending the session.

4. API Endpoints

Method	Endpoint	Description	Source	Stage
POST	`/auth/login`	Authenticate user, return session token + role	UC-01	3
POST	`/auth/logout`	End user session / invalidate token	UC-02	3
GET	`/rooms`	List all rooms	—	1
POST	`/rooms`	Add a room (Admin only)	—	2
DELETE	`/rooms/{room_id}`	Delete a room (reject if devices still assigned)	—	3
GET	`/devices`	List devices (paginated, filterable, sortable)	Activity #5	1
GET	`/devices/{device_id}`	Get single device detail (for edit pre-fill)	—	3
POST	`/devices`	Add a device (Admin only)	Lab #9	2
PUT	`/devices/{device_id}`	Update a device	Lab #10	3
DELETE	`/devices/{device_id}`	Delete a device (Admin only)	Lab #10	3

Method	Endpoint	Description	Source	Stage
GET	`/readings`	List readings (paginated, filterable)	Activity #5	2
POST	`/readings`	Add reading (auto-checks thresholds → alert)	Lab #9	2
GET	`/alerts`	List alerts (filterable by acknowledged status)	—	2
PUT	`/alerts/{alert_id}/acknowledge`	Acknowledge an alert	—	3
GET	`/dashboard/summary`	Avg PM2.5, CO2, device count, alert count	—	2
GET	`/dashboard/chart`	Last 24h of readings for line chart	—	3
GET	`/export/readings`	Export readings as CSV	FR-21	3

---

5. Flet Page & Control Mapping

Page	Flet Controls	Notes	Stage
Login	TextField (username/password)	Entry point. Redirects to Dashboard on success.	3

Page	Flet Controls	Notes	Stage
	, ElevatedButton, Snackbar, Text		
Dashboard	AppBar, MenuBar, Row of 4 summary Cards, LineChart, BottomAppBar, BottomSheet	Auto-refresh every 5s via `asyncio.sleep`	2 / 3
Devices / Inventory	AppBar, TextField (search), Dropdown (sort/order), DataTable, Pager Row, BottomSheet (add), AlertDialog (edit/delete), counter Text	Full CRUD for Admin. `page.dialog = edit_dialog` MUST be registered or dialog won't render.	1 / 2 / 3
Alerts	AppBar, DataTable, Dropdown (filter), IconButton (acknowledge)	Viewable by all roles	2 / 3
Settings	AppBar, Dropdown (select device), TextField (PM2.5/CO2 limits), ElevatedButton (save), Export CSV button	Admin only	3

---

6. Validation Rules

6.1 Field Validation

Field	Frontend (Flet)	Backend (Pydantic)
device_id	Required, not empty	`min_length=1, pattern="^[A-Za-z0-9_-]+$"`
model	Required, not empty	`min_length=1`
room_id	Required, selected from dropdown	Must exist in rooms table
pm25	Required, numeric, >= 0	`ge=0`
co2	Required, numeric, >= 0	`ge=0`
threshold	Required, numeric, >= 0	`ge=0`
Search input	Min 1 char to trigger	Sanitized (LIKE with %%)
username	Required, min 3 chars	`min_length=3, max_length=50`, alphanumeric
password	Required, min 8 chars, shown as dots	Validated before bcrypt hash
role (login response)	Read from API response after login; store in page state	API returns role field in login response

6.2 Critical Implementation Notes (from Lab #9, #10, Activity #5)

 These are the most common coding mistakes flagged by the labs. Read before writing any code.

#	Rule	Source	Why it matters
1	`page.dialog = edit_dialog` MUST be assigned before calling	Lab #10	Forgetting this means the AlertDialog never renders — no error

#	Rule	Source	Why it matters
	`edit_dialog.open = True`		shown, just silent failure
2	`current_page` must be stored as a list: `current_page = [1]`, not a plain int	Activity #5	Python closures can read outer variables but cannot reassign them. Using `current_page[0] = n` mutates in-place so all inner functions share state
3	Use closure wrappers `make_delete(iid)` and `make_edit(row_data)` inside the table loop	Lab #10	Without closures, every button captures LAST loop variable, so every button acts on the last row
4	Pagination offset formula: `offset = (current_page[0] - 1) * PAGE_SIZE`	Activity #5	Off-by-one error. Using `current_page[0] * PAGE_SIZE` skips page 1
5	Reset `current_page[0] = 1` whenever the search term changes	Activity #5	If user on page 5 types a search returning only 2 pages, they see 0 results
6	Always call `rebuild_pager()` and `page.update()` at the END of `load_table()`	Activity #5	Forgetting this leaves stale page numbers after filtering or sorting

7. File Structure & Build Order

7.1 File Structure

air_quality_project/

```
├──── PLAN.md            ← This document
├──── requirements.txt   ← pip dependencies
├──── config.py          ← Shared constants (DB path, API URL, thresholds)
├──── models.py          ← Pydantic models (Device, Reading, Alert, Room, User)
├──── api.py             ← FastAPI + SQLite3 (all endpoints)
├──── seed.py            ← Seed: rooms, devices, readings, alerts, users
├──── main.py            ← Flet app (Login + 4 main pages)
├──── simulator.py       ← Background fake sensor data generator
└──── air_quality.db     ← SQLite DB (auto-created on first run)
```

7.2 Build Order

 `models.py` must be built BEFORE `api.py` because `api.py` imports from it.

Step	File	What to Build
1	`requirements.txt`	`fastapi`, `uvicorn`, `flet`, `pydantic`, `bcrypt`, `python-multipart`, `requests`
2	`config.py`	DB_PATH, API_BASE_URL, default threshold constants

Step	File	What to Build
3	`models.py`	Pydantic models with validators (BEFORE api.py)
4	`api.py`	SQLite schema creation + all CRUD + auth + export endpoints
5	`seed.py`	5 rooms, 10 devices, 100+ readings, 2 users with bcrypt-hashed passwords
6	`main.py`	Login page + 4 main pages, NavigationBar, all Flet controls
7	`simulator.py`	Background loop: every 5–10s POST a random reading

□ BUILD STRATEGY: The 4-Stage Approach

Sections 8–12 describe how to **actually build** the project across 4 progressive stages that match your class curriculum (Lab 9 → Lab 10 → Activity 5 → Lab 19). After each stage, you have a **working app** you could submit. The same files (config.py, api.py, main.py, etc.) get progressively upgraded — and in Stage 4 the entire stack is containerised with Docker.

---

8. STAGE 1 — Lab 9 (1.5 days)

Goal: A working FastAPI + SQLite3 server with a POST endpoint, and a Flet app that submits records and shows them in a DataTable, with navigation between two screens via NavigationBar. **End state:** Even if you stop here, you have a passable submission. ✓

8.1 What You'll Learn

Concept	Where used
SQLite basics (CREATE TABLE, INSERT, SELECT)	Database
FastAPI GET + POST endpoints	Backend
Pydantic BaseModel for input validation	Models
HTTP POST — sending JSON data from Flet	Frontend
requests.post() with JSON payload	Frontend
Flet DataTable (DataColumn, DataRow)	UI
NavigationBar with visible=True/False	UI
Snackbar for success/error feedback	UI

8.2 Features in Stage 1

Only these FRs: **FR-04, FR-08, FR-11, FR-16, FR-20** (DataTable + POST form + NavigationBar + Snackbar + SQLite persistence)

8.3 Files to Build (Stage 1)

config.py ← DB_PATH, API_BASE_URL

models.py ← DeviceCreate (basic Pydantic)

api.py ← GET /devices, POST /devices (SQLite3 init_db + get_connection)

seed.py ← Creates DB + 5 rooms + 10 devices

main.py ← TWO views: Records table + Add New form, NavigationBar

8.4 Stage 1 Testing Checklist

#	Test	Pass?
1	`python seed.py` creates DB with rooms + devices	☐
2	`uvicorn api:app --reload --port 8000` starts without errors	☐
3	Visit `http://127.0.0.1:8000/docs` — see GET and POST /devices	☐
4	POST `/devices` via Swagger returns `{ "message": "Record added successfully" }`	☐
5	GET `/devices` returns full JSON array	☐
6	`python main.py` — NavigationBar with Records and Add New tabs appears	☐
7	Fill Add New form + submit → device appears in table + green Snackbar	☐
8	Submit with any empty field → red Snackbar, no POST sent	☐
9	Submit a duplicate ID → red Snackbar error from SQLite	☐

#	Test	Pass?
10	Restart app → all data still there (SQLite persistence)	☐

✅Stage 1 complete when all 10 tests pass.

---

9. STAGE 2 — Lab 10 (1.5 days)

Goal: Extend Stage 1 with DELETE and PUT endpoints, per-row Edit and Delete buttons in Flet, an edit modal dialog, and a live search field. Full CRUD cycle now complete.

9.1 What You'll Learn

Concept	Where used
HTTP DELETE — remove a resource by ID	API
HTTP PUT — replace an entire record	API
FastAPI path parameters `{device_id}`	API
AlertDialog in Flet (edit modal)	UI
Closure wrappers `make_delete` / `make_edit`	UI
GET with `?search=` query parameter	API
Live search via on_change → load_table()	UI
IconButton for per-row actions	UI

9.2 Features in Stage 2

New FRs: **FR-06, FR-09, FR-10** (live search + edit dialog + delete with confirmation) All Stage 1 features continue to work.

9.3 Files Updated in Stage 2

api.py ← Add: PUT /devices/{id}, DELETE /devices/{id},

update GET /devices to accept ?search= query param

main.py ← Add: Edit IconButton + AlertDialog per row,

Delete IconButton with make_delete closure,

Search TextField with on_change → load_table

9.4 Key Concept: Closures in Loops

Without a closure wrapper every button in the table acts on the last row only:

```
# ✕Wrong — all buttons capture the last item_id
for item in data:
    ft.IconButton(on_click=lambda e: delete(item["id"]))

# ✓Correct — closure captures the current item_id
def make_delete(iid):
    def on_delete(e):
        requests.delete(f"{API_URL}/devices/{iid}")
    return on_delete
```

9.5 Stage 2 Testing Checklist

#	**Test**	**Pass?**
1	All 10 Stage 1 tests still pass	☐
2	Swagger shows DELETE and PUT for `/devices/{device_id}`	☐
3	DELETE via Swagger removes the device; error returned if ID not found	☐

#	Test	Pass?
4	PUT via Swagger updates a device; change visible in GET	☐
5	Each table row shows Edit (pencil) and Delete (trash) icon buttons	☐
6	Click Delete → green Snackbar + row disappears	☐
7	Click Edit → modal dialog opens with pre-filled fields	☐
8	Change a field + Save → updated value appears in table + Snackbar	☐
9	Type in Search field → table filters in real time	☐
10	Restart app → all remaining records persist in SQLite3	☐

✅Stage 2 complete when all 10 tests pass.

---

10. STAGE 3 — Activity 5 (1.5 days)

Goal: Extend the API and Flet UI with server-side pagination, text search, and column sorting. The table now loads only one page at a time with dynamic Previous / Next / page-number controls.

10.1 What You'll Learn

Concept	Where used
SQL LIMIT / OFFSET for page slicing	API
COUNT(*) for total rows (needed for page math)	API
Query parameters: `limit`, `offset`, `search`, `sort_by`, `order`	API
Column whitelist to prevent SQL injection via sort_by	API
`current_page = [1]` list trick for mutable closure state	UI
Rebuilding a dynamic pager Row on every load	UI
Sort Dropdown + Order Dropdown	UI
Counter label "Showing X–Y of Z records"	UI

10.2 Features in Stage 3

New FRs: **FR-05, FR-07** (pagination + column sorting) FR-06 (search) is upgraded from client-side to server-side. All Stage 1 + Stage 2 features continue to work.

10.3 Files Updated in Stage 3

api.py ← Replace GET /devices with paginated version:

accepts limit, offset, search, sort_by, order

returns { total, limit, offset, items: [...] }

main.py ← Add: sort_dropdown, order_dropdown, search_field,

counter_text, pager_row, rebuild_pager(),

go_to_page(), updated load_table() with offset math

seed.py ← Add: run once to insert 100 sample devices for paging test

10.4 Key Concept: Pagination Formula

```
# offset = rows to skip before the current page

offset = (current_page[0] - 1) * PAGE_SIZE

# total_pages = how many pages exist

total_pages[0] = math.ceil(total / PAGE_SIZE)

# ✕Wrong — off by one: page 1 skips 10 rows

offset = current_page[0] * PAGE_SIZE
```

10.5 Stage 3 Testing Checklist

#	**Test**	**Pass?**
1	All 20 Stage 1 + Stage 2 tests still pass	☐
2	Run seed to insert 100 devices; app shows only 10 at a time	☐
3	Counter reads "Showing 1–10 of 100 records" on page 1	☐
4	Click page 3 → counter changes to "Showing 21–30 of 100"	☐
5	Click Next repeatedly → button disables on last page	☐
6	Type in Search → pager resets to page 1, matching rows shown	☐

#	Test	Pass?
7	Change Sort by to model, Order to Descending → table reorders	☐
8	Clear search → all 100 records return across 10 pages	☐
9	Swagger: `GET /devices?limit=5&offset=10&sort_by=model&order=desc` works	☐
10	Swagger: `sort_by=malicious_col` falls back silently to default	☐

✅Stage 3 complete when all 10 tests pass.

---

11. STAGE 4 — Lab 19 / Docker (1 day)

Goal: Package the entire stack (FastAPI backend + Flet frontend + SQLite3 volume) into Docker containers and wire them together with docker-compose. The full app starts with one command and survives restarts.

11.1 What You'll Learn

Concept	Where used
Docker image vs container	Infrastructure
Dockerfile instructions (FROM, WORKDIR, COPY, RUN, CMD, EXPOSE)	Infrastructure
Layer caching — copy requirements.txt before source code	Dockerfile

Concept	Where used
Named volumes — SQLite3 data survives `docker compose down`	docker-compose
Docker networking — containers reach each other by service name	docker-compose
Environment variables via `.env` file	Both
Flet web mode (`view=ft.AppView.WEB_BROWSER`) for containerised use	Frontend

11.2 Features in Stage 4

FR-20 is now satisfied at the infrastructure level — data persists via a named Docker volume even if the container is removed. All Stage 1–3 features continue to work, now running inside containers.

11.3 Project Restructure for Stage 4

air_quality_project/

├── backend/

│ ├── api.py

│ ├── requirements.txt

│ └── Dockerfile

├── frontend/

│ ├── main.py

│ ├── requirements.txt

│ └── Dockerfile

├── data/ ← SQLite .db file lives here (volume mount)

├── .env ← API_HOST=backend, API_PORT=8000, DB_PATH=/data/air_quality.db

└── docker-compose.yml

11.4 Key Changes to Existing Files

backend/api.py — read DB path from environment:

```
import os

DB_NAME = os.getenv("DB_PATH", "air_quality.db")
```

frontend/main.py — read API host from environment and run in web mode:

```
import os

API_HOST = os.getenv("API_HOST", "127.0.0.1")

API_PORT = os.getenv("API_PORT", "8000")

API_URL  = f"http://{API_HOST}:{API_PORT}"

# ...

ft.app(target=main, view=ft.AppView.WEB_BROWSER, port=8550)
```

.env:

```
API_HOST=backend

API_PORT=8000

DB_PATH=/data/air_quality.db
```

11.5 Stage 4 Testing Checklist

#	Test	Pass?
1	All 30 Stage 1–3 tests still pass (run locally before containerising)	☐
2	`docker compose build` completes without errors for both services	☐

#	Test	Pass?
3	`docker compose up` — both containers appear healthy in the log	☐
4	`http://localhost:8000/docs` opens FastAPI Swagger UI	☐
5	`http://localhost:8550` opens the Flet app in the browser	☐
6	Add a device via the Flet UI — it appears in the table	☐
7	Call `GET /devices` in Swagger — the same device is returned	☐
8	`docker compose down` then `docker compose up` — device still exists (volume persists)	☐
9	`docker compose down -v` then `docker compose up` — data is gone (volume deleted)	☐
10	`docker compose logs backend` — API requests visible in output	☐

✅ Stage 4 complete when all 10 tests pass. PROJECT DONE! 🎉

---

12. Build Timeline & Rules

12.1 Timeline

Stage	Lab / Activity	Time	Cumulative
Stage 1	Lab 9	1.5 days	1.5 days
Stage 2	Lab 10	1.5 days	3 days
Stage 3	Activity 5	1.5 days	4.5 days
Stage 4	Lab 19 (Docker)	1 day	**5.5 days total**

 Safety net: If your deadline tightens, prioritize completing each stage fully rather than rushing partway into the next. A complete Stage 3 beats a half-broken Stage 4.

12.2 Universal Rules (All Stages)

1. **No new features beyond the FR list** — stick to the plan
2. **No perfectionism** — working > beautiful (polish at end)
3. ✅**Test after each stage** — don't move forward with broken code
4. ✅**Use parameterized SQL queries** — no f-strings in SQL ever
5. ✅**Validate on both frontend AND backend** — defense in depth
6. ✅**Use try/except + Snackbar** for all API calls in the UI
7. ✅**Take 10-min breaks every 2 hours** — your brain needs it

---

13. Definition of Done

The project is complete when:

- ✅All Stage 1 tests pass (10 tests — POST + NavigationBar + SQLite)
- ✅All Stage 2 tests pass (10 tests — DELETE + PUT + Edit dialog + Search)
- ✅All Stage 3 tests pass (10 tests — pagination + sort + filter)
- ✅All Stage 4 tests pass (10 tests — Docker + docker-compose + volumes)
- ✅All 22 FRs implemented

- ✅All 7 NFRs satisfied
- ✅Admin and Operator roles enforced (RBAC working)
- ✅README.md explains how to run the project

Legend

Symbol	Meaning
✅	Complete / passes
☐	To do
⚠️	Critical warning — read carefully
🚫	Do not do this

End of Plan Document SE Project 777 | Complete Plan (Academic + 4-Stage Build)